



CELEBRATING 50 YEARS

Dilip Kumar Maripuri
Computer Applications





Data Structures

Session : Expression Trees

Dilip Kumar Maripuri
Computer Applications

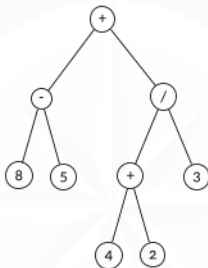


Data Structures

Expression Trees

- ▶ An **expression tree** is a binary tree used to represent mathematical expressions. In such a tree:
 - ▶ Leaf nodes contain operands (constants or variables).
 - ▶ Internal nodes contain operators.
- ▶ Expression trees allow efficient expression evaluation & transformation.





Representation of the expression $(8 - 5) * ((4 + 2) / 3)$ in the form Expression Tree



1. Infix Notation ($A + B$)

2. Postfix Notation (Reverse Polish Notation: A B +)

3. Prefix Notation (Polish Notation: + A B)

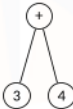


Data Structures

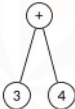
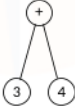
Building Expression Trees

- ▶ Expression trees are typically constructed using Postfix Notation
 1. Read the expression token by token.
 2. If the token is an operand, create a node and push it onto a stack.
 3. If the token is an operator:
 - ▶ Pop two nodes from the stack.
 - ▶ Make them the children of the operator node.
 - ▶ Push the operator node back onto the stack.
 4. The remaining node in the stack is the root of the expression tree.

 $34 + 52 - *$

Scanned Char	Action	Snapshot
3	Create a node with 3	3
4	Create a node with 3	4 3
+	Create a node with + Perform POP twice  <pre> graph TD A((+)) --- B((3)) A --- C((4)) </pre> Push + node onto Stack	+

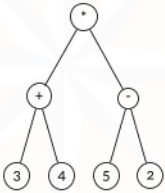
$34 + 52 - *$

Scanned Char	Action	Snapshot
5	Create Node with 5 and push onto Stack 	
2	Create Node with 2 and push onto Stack 	

 $34 + 52 - *$

Scanned Char	Action	Snapshot
-	Create Node with - and perform POP twice  Push node with - onto the stack	

 $34 + 52 - *$

Scanned Char	Action	Snapshot
*	Create Node with * and perform POP twice  Push node with * onto the stack	

 $34 + 52 - *$

Scanned Char	Action	Snapshot
NULL	Perform POP operation and Store in Root 	EMPTY



```
1 typedef struct Node {
2     char data;
3     struct Node *left, *right;
4 } *Node;
5
6 typedef struct Stack {
7     Node data;
8     struct Stack* next;
9 } *Stack;
10
11 Node createNode(char data) {
12     Node newNode = (Node)malloc(sizeof(struct Node));
13     newNode->data = data;
14     newNode->left = newNode->right = NULL;
15     return newNode;
16 }
```



```

1 Stack push(Stack top, Node node) {
2     Stack newStackNode = (Stack)malloc(sizeof(struct Stack));
3     ;
4     newStackNode->data = node;
5     newStackNode->next = top;
6     return newStackNode;
7 }
8
9 Stack pop(Stack top, Node* poppedNode) {
10     if (top == NULL) return NULL;
11     *poppedNode = top->data;
12     Stack temp = top;
13     top = top->next;
14     free(temp);
15     return top;
16 }

```



```

1 Node buildExpressionTree(char postfix[]) {
2     Stack stack = NULL;
3     for (int i = 0; postfix[i] != '\0'; i++) {
4         if (isdigit(postfix[i]))
5             stack = push(stack, createNode(postfix[i]));
6         else {
7             Node right, left;
8             stack = pop(stack, &right);
9             stack = pop(stack, &left);
10            Node newNode = createNode(postfix[i]);
11            newNode->right = right;
12            newNode->left = left;
13            stack = push(stack, newNode);
14        }
15    }
16    Node root;
17    stack = pop(stack, &root);
18    return root;
19 }

```

 Dilip Kumar



```
1 int main() {  
2     char postfix[] = "34+52-*";  
3     Node root = buildExpressionTree(postfix);  
4     inorder(root);  
5 }
```



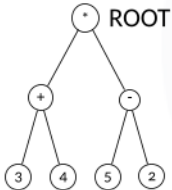

Data Structures

Evaluation of Expression Trees



- ▶ To evaluate an expression tree:
 1. Recursively evaluate the left subtree.
 2. Recursively evaluate the right subtree.
 3. Apply the operator at the root to the values from the left and right subtrees.



Expression Tree	Step No	Subtree Evaluated	Operation	Result
	1	3 (Leaf)	Return value	3
	2	4 (Leaf)	Return value	4
	3	$3 + 4$	Addition	7
	4	5 (Leaf)	Return value	5
	5	2 (Leaf)	Return value	2
	6	$5 - 2$	Subtraction	3
	7	$7 * 3$	Multiplication	21



Dilip Kumar Maripuri



Data Structures

Practise Questions

- For each of the given postfix expressions, construct the corresponding expression tree.

1. $23+$
2. $34+5^*$
3. $12+34-^*$
4. $56+72-/$
5. $23+45+^*67-8/+$
- 6.



Data Structures

Practise Questions

- Given the following infix expressions, first convert them to postfix notation, then construct the expression tree.

1. $(A + B) * C$
2. $(X - Y) / (Z + W)$
3. $(P + Q) * (R - S) / T$
4. $A + B * C - D / E$
5. $((M + N) * O) / P - Q$



Data Structures

Practise Questions

- Evaluate the following postfix expressions by constructing an expression tree first, then computing the result.

1. $34+$
2. $52-$
3. $34+5^*$
4. $72/56+^*$
5. $34+52-^*$



Thank You

Dilip Kumar Maripuri
Associate Professor
Department of Computer Applications
dilip.maripuri@pes.edu
8073212026